

METU Computer Engineering

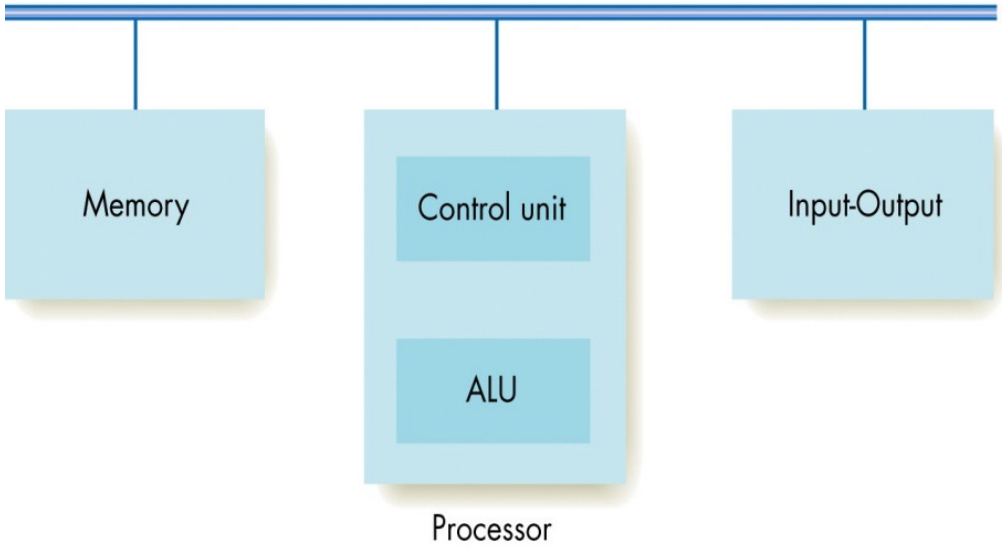
CEng 111 – Fall 2026
Week 3b

Credit: Some slides are from the “Invitation to Computer Science” book by G. M. Schneider, J. L. Gersting and some from the “Digital Design” book by M. M. Mano and M. D. Ciletti.

Previously on CENG111!

Central Processing Unit (CPU)

- Control Unit
- Arithmetic Logic Unit (ALU)



Previously on CENG111!

The Control Unit

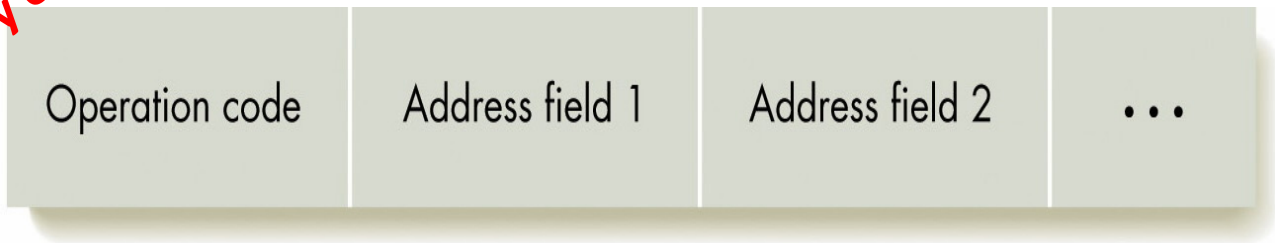


- Manages stored program execution
- Task

1. Fetch from memory the next instruction to be executed
2. Decode it: determine what is to be done
3. Execute it: issue appropriate command to ALU, memory, and I/O controllers

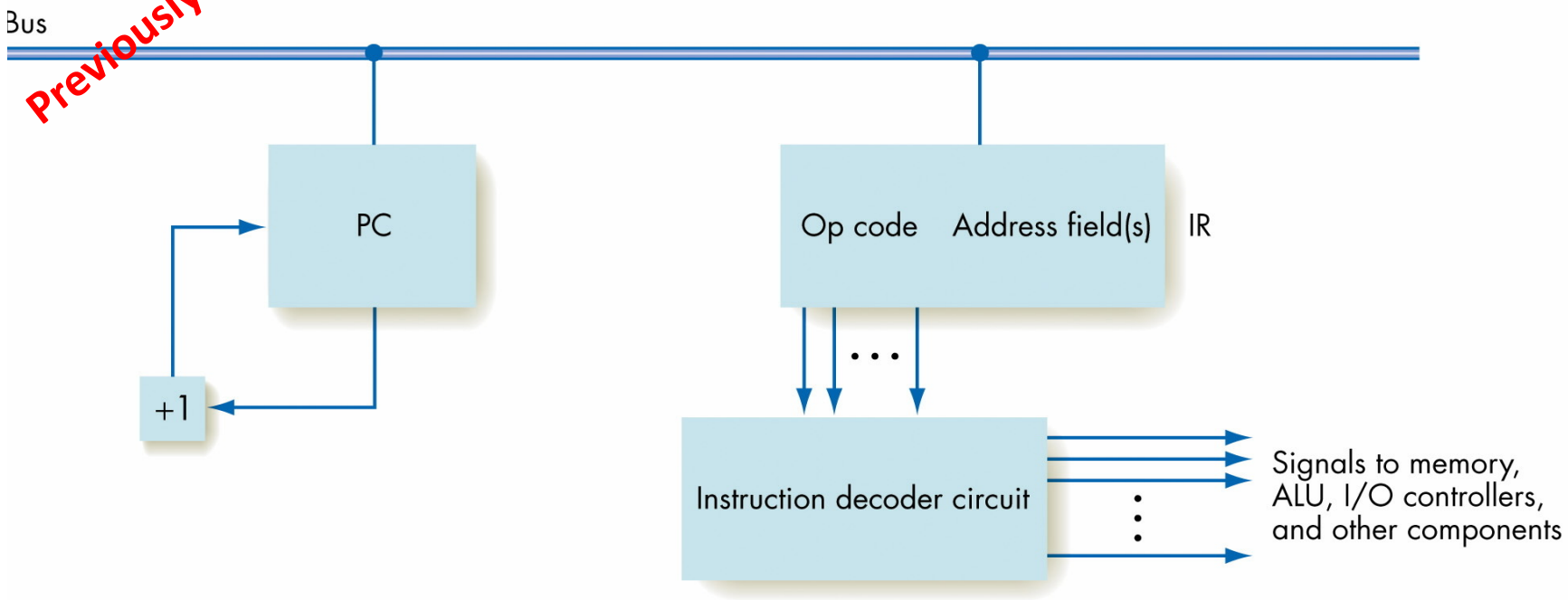
Previously on CENG111!

More on Instructions



- LOAD X -> Load register R with the contents of memory cell X
- STORE X -> Store register R into memory cell X
- MOVE X, Y -> Copy the contents of X into Y
- ADD X -> Add contents of X to the contents of R
- ADD X,Y -> Add contents of X to the contents of Y, and put the result in register R
- COMPARE X, Y-> Set GT (greater than), EQ (equal) and LT (less than) condition codes
- JUMP X-> Jump unconditionally to the instruction in cell X
- JUMPGT X-> Jump, if GT=1, to the instruction in cell X
- HALT

Previously on CENG111!



Organization of the Control Unit Registers and Circuits



Today

- Continue with the Von Neumann Architecture
  - CPU
    - Control Unit and the Fetch-Decode-Execute Cycle
    - Arithmetic Logic Unit



Administrative Notes

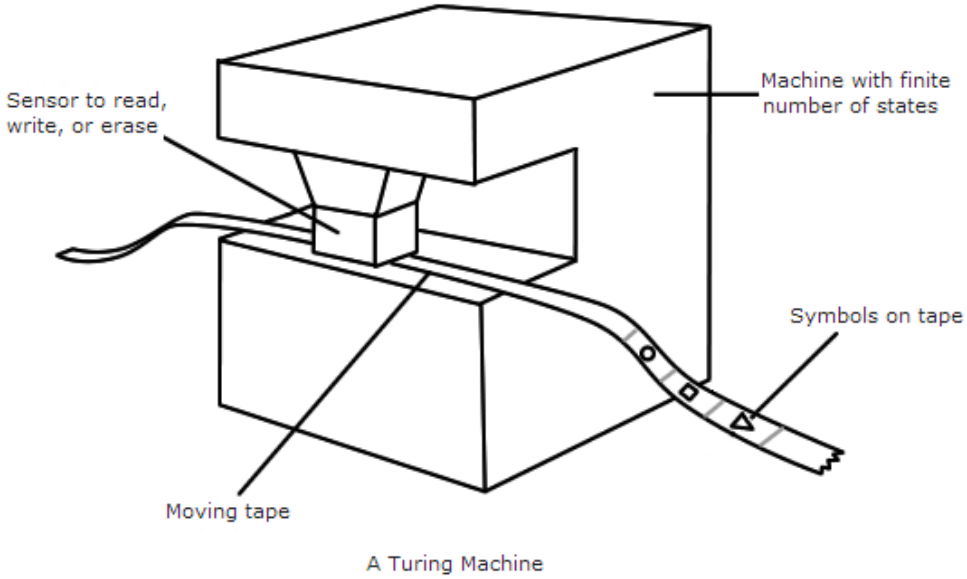
- Labs are starting this week!
- Von Neumann Machine Exam:
  - Preliminary Text has been announced!
  - In-class Exam: 23 October (Friday) – 18:00

Turing Machine



Alan Turing
(1912-1954)

- Main components:
  - Read-write head
  - Infinite memory tape
  - Rule book



A Turing Machine

- It has a set of states and it can recognize a set of symbols

Turing Machine



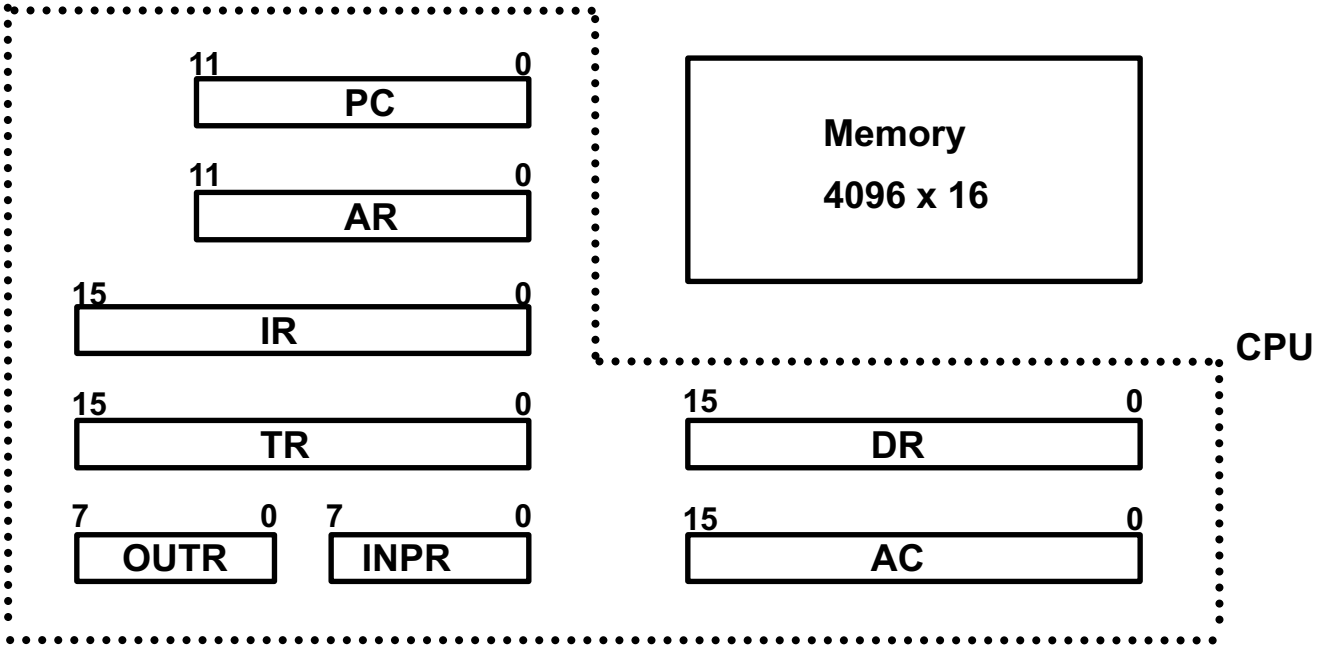
Alan Turing
(1912-1954)

- Follows these steps:
  - Read current symbol on the tape
  - Look at the rule book for that symbol
  - Execute the corresponding rule
- Example (from the second textbook):

State now	Read symbol	Write symbol	Move tape	Next state
A	0	1	RIGHT	**B**
A	1	1	LEFT	**C**
B	0	1	LEFT	**A**
B	1	1	RIGHT	**B**
C	0	1	LEFT	**B**
C	1	1	RIGHT	**HALT**

METU Computer Engineering

Registers in a Basic Computer



List of BC Registers

DR	16	Data Register	Holds memory operand
AR	12	Address Register	Holds address for memory
AC	16	Accumulator	Processor register
IR	16	Instruction Register	Holds instruction
PC	12	Program Counter	Holds address of instruction
TR	16	Temporary Register	Holds temporary data
INPR	8	Input Register	Holds input character
OUTR	8	Output Register	Holds output character

Main Types of Registers

- **User-accessible Registers**:
  - The most common division of user-accessible registers is into data registers and address registers.
- **Data registers**: hold numeric values such as integer and floating-point values.
  - In some older and low end CPUs, a special data register, known as the accumulator, is used implicitly for many operations.
- **Address registers:** hold addresses and are used by instructions that indirectly access memory.
- **Conditional registers:** hold truth values often used to determine whether some instruction should or should not be executed.
- **General purpose registers (GPRs)** can store both data and addresses, i.e., they are combined Data/Address registers.

http://en.wikipedia.org/wiki/Processor_register

Main Types of Registers (cont'd)

- **Floating point registers** (FPRs) store floating point numbers in many architectures.
- **Constant registers** hold read-only values such as zero, one, or pi.
- **Special purpose registers** (SPR) hold program state; they usually include the program counter (aka instruction pointer), stack pointer, and status register (aka processor status word). In embedded microprocessors, they can also correspond to specialized hardware elements.
- **Instruction registers** store the instruction currently being executed.

http://en.wikipedia.org/wiki/Processor_register

METU Computer Engineering



Some Architectures & The number of registers on them

Architecture	Integer registers	FP registers
x86	8	8
x86-64	16	16
IBM/360	16	4
Z/Architecture	16	16
Itanium	128	128
UltraSPARC	32	32
IBM POWER	32	32
Alpha	32	32
6502	3	0
PIC microcontroller	1	0
AVR microcontroller	32	0
ARM	16	16

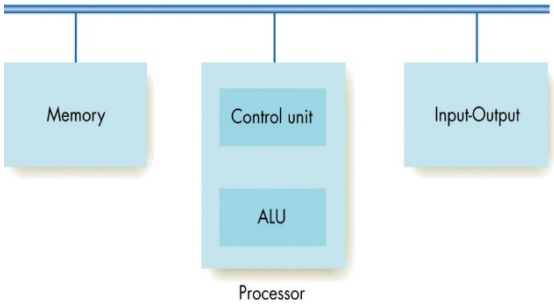
http://en.wikipedia.org/wiki/Processor_register

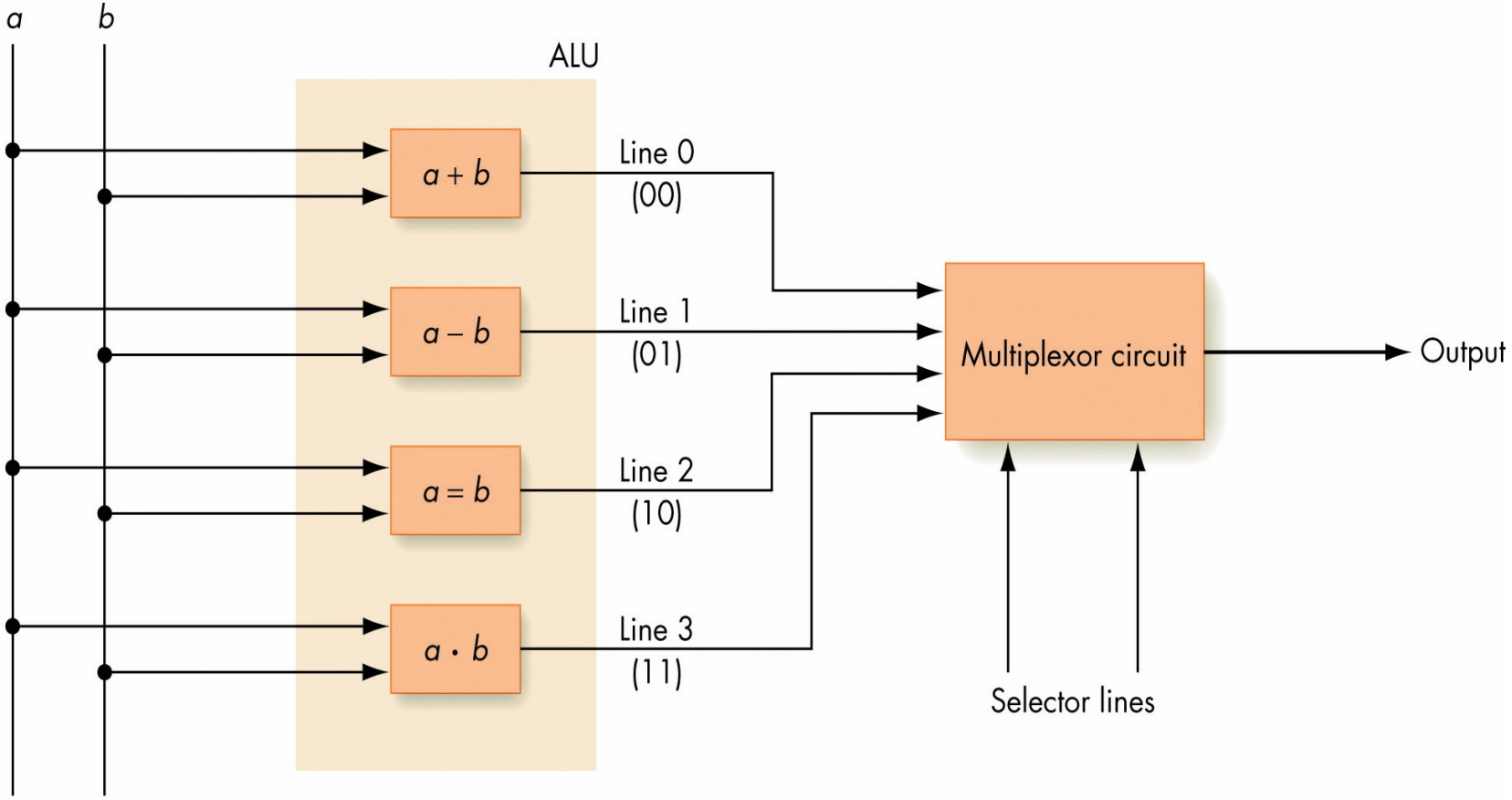
METU Computer Engineering

The Arithmetic/Logic Unit

METU Computer Engineering

- ***A**rithmetic and **L**ogic **U**nit*
  - *"Manufacturing" section of computer*
  - *Contains decision mechanisms and can make calculations+comparisons*
  - Actual computations are performed
- Primitive operation circuits
  - Arithmetic *[+, -, *, /]*
  - Comparison *[equality or CE, GT, LT, NEQ]*
  - Logic *[AND, OR, NOT, XOR]*
- Data inputs and results stored in registers
- Multiplexer selects desired output





Using a Multiplexor Circuit to Select the Proper ALU Result



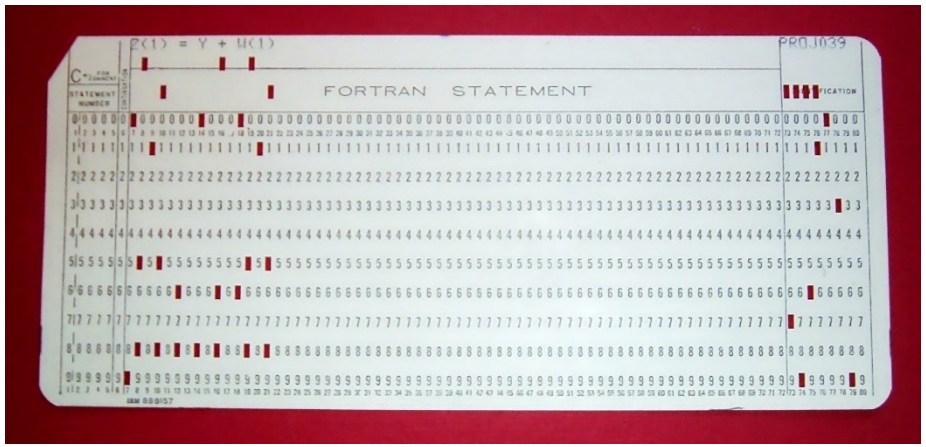
The Arithmetic/Logic Unit (continued)

- ALU process
  - Values for operations copied into ALU's input register locations
  - All circuits compute results for those inputs
  - Multiplexor selects the one desired result from all values
  - Result value copied to desired result register

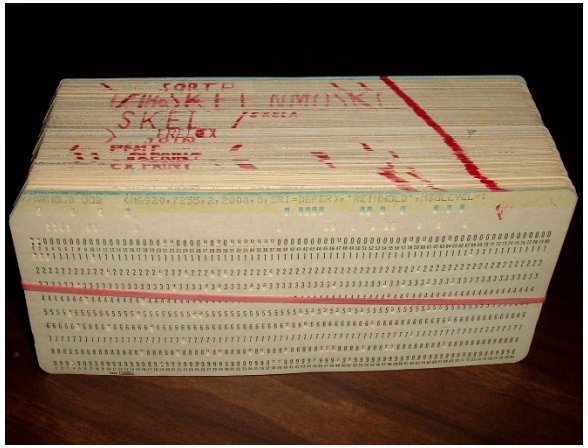


MORE ON
THE VON NEUMANN ARCHITECTURE

The Era of Punched Cards







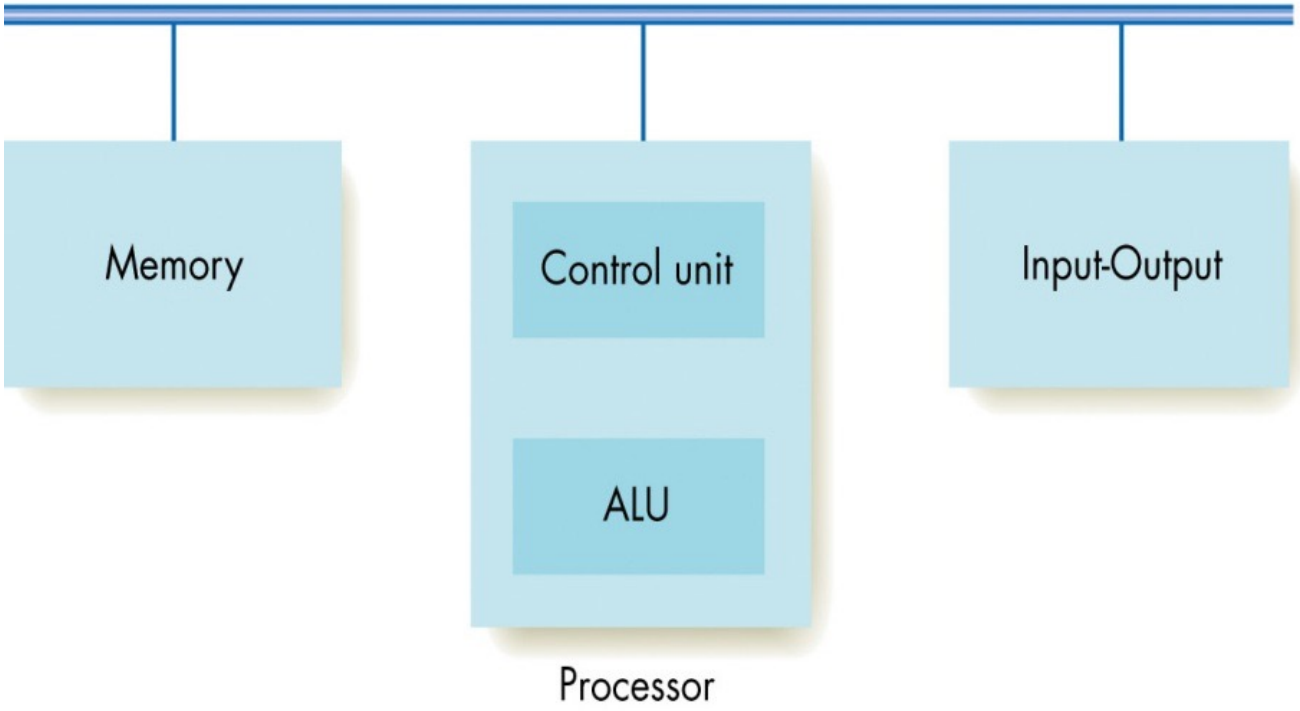
https://en.wikipedia.org/wiki/Computer_programming_in_the_punched_card_era

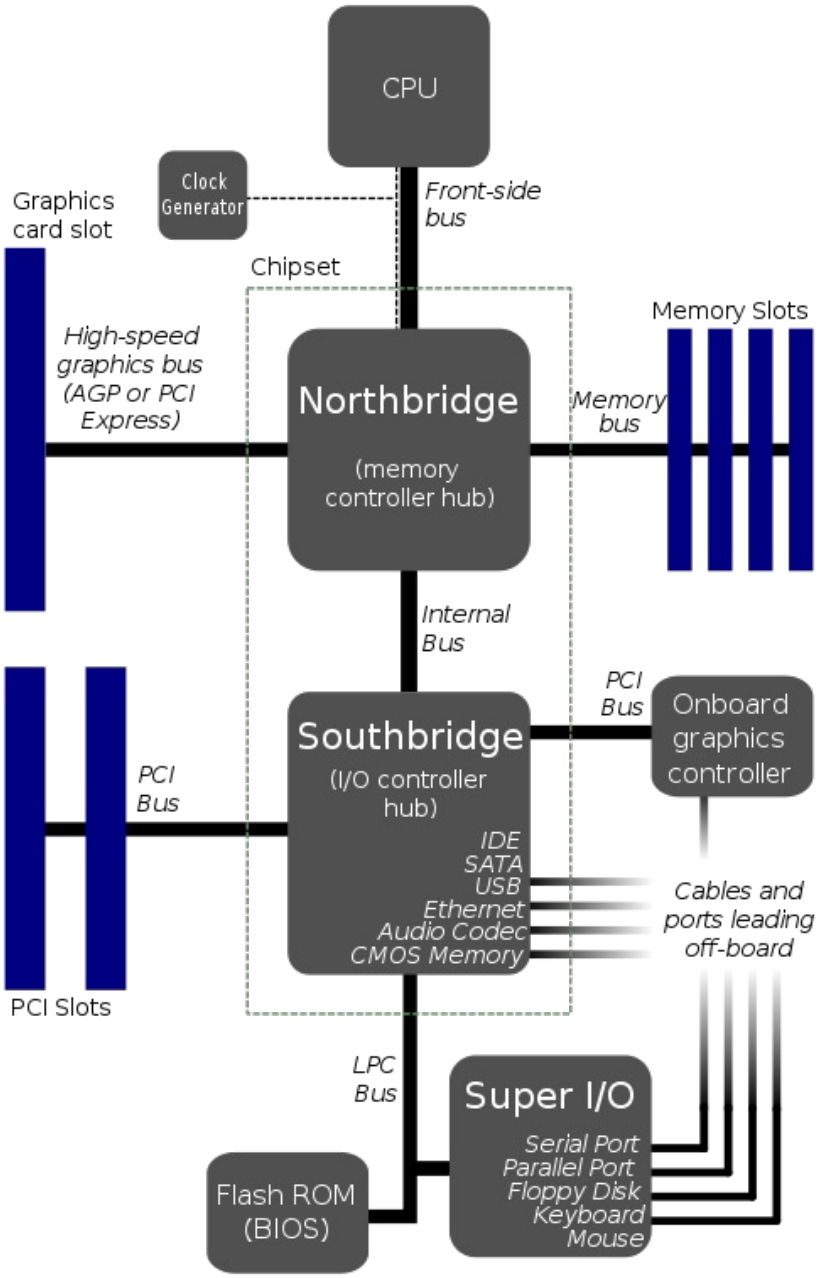
METU Computer Engineering



The Era of Punched Cards

- Machines:
  - Have small memory
  - Some do not have ALU
  - Read programs from the punched card
  - You submit your punch cards, get into a queue and wait for the output
- Example machine:
  - https://en.wikipedia.org/wiki/IBM_1620


Memory
Control unit
ALU
Input-Output
Processor



https://en.m.wikipedia.org/wiki/File:Motherboard_diagram.svg

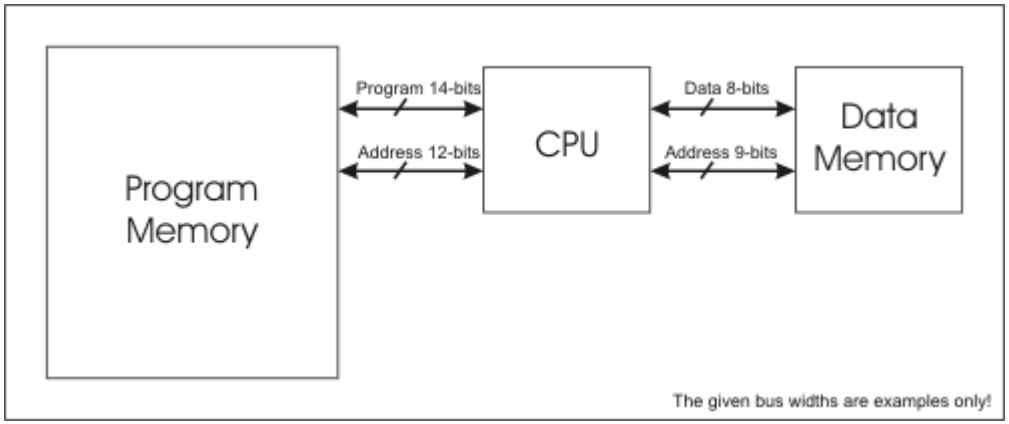
Von Neumann Architecture

- **Pros:**
  - Simplifies hardware (both circuit-design and layout)
  - Easier to generate re-locatable code, which makes multi-tasking easier to implement.

- **Cons:**
  - Instructions must be multiples of the data bus-width - can be inefficient.
  - Variable number of cycles required for instructions. For example, an instruction that requires data from memory must wait at least another cycle before it can complete, whereas some instructions execute much faster. This can be a problem for time-critical applications.

http://www.mhennessy.f9.co.uk/pic/architecture.htm

Harvard Architecture



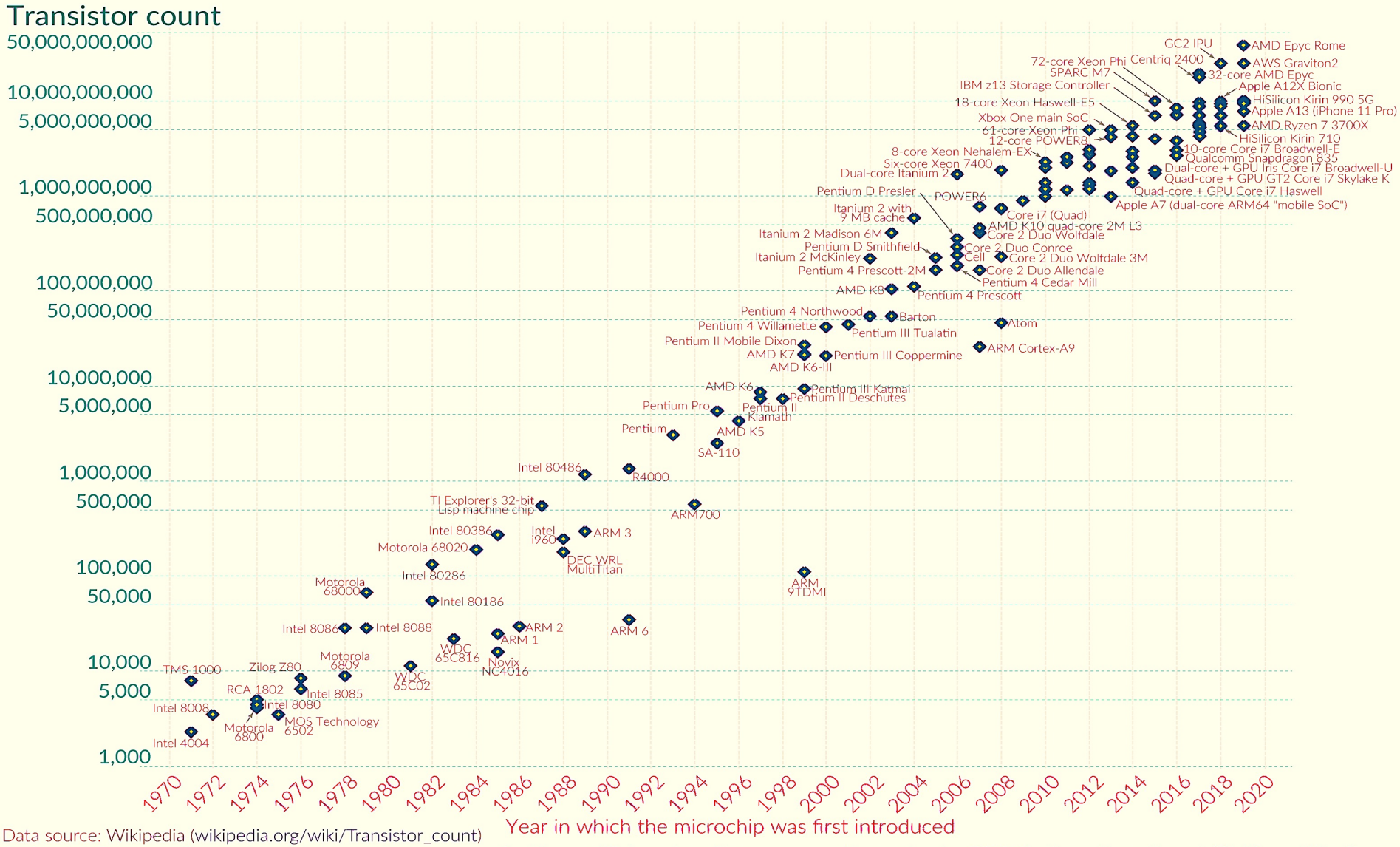
http://www.mhennessy.f9.co.uk/pic/architecture.htm

- **Pros:**
  - Data and address busses can be of different widths. This means the program memory word can be wide enough to incorporate an instruction and a literal (fixed data) in a single instruction.
  - A built-in two-stage pipeline overlaps fetch and execution of instructions, meaning most instructions execute in a single clock cycle.
- **Cons:**
  - Slightly more confusing.
  - Hardware is more complicated.

Moore's Law: The number of transistors on microchips doubles every two years

Our World in Data

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

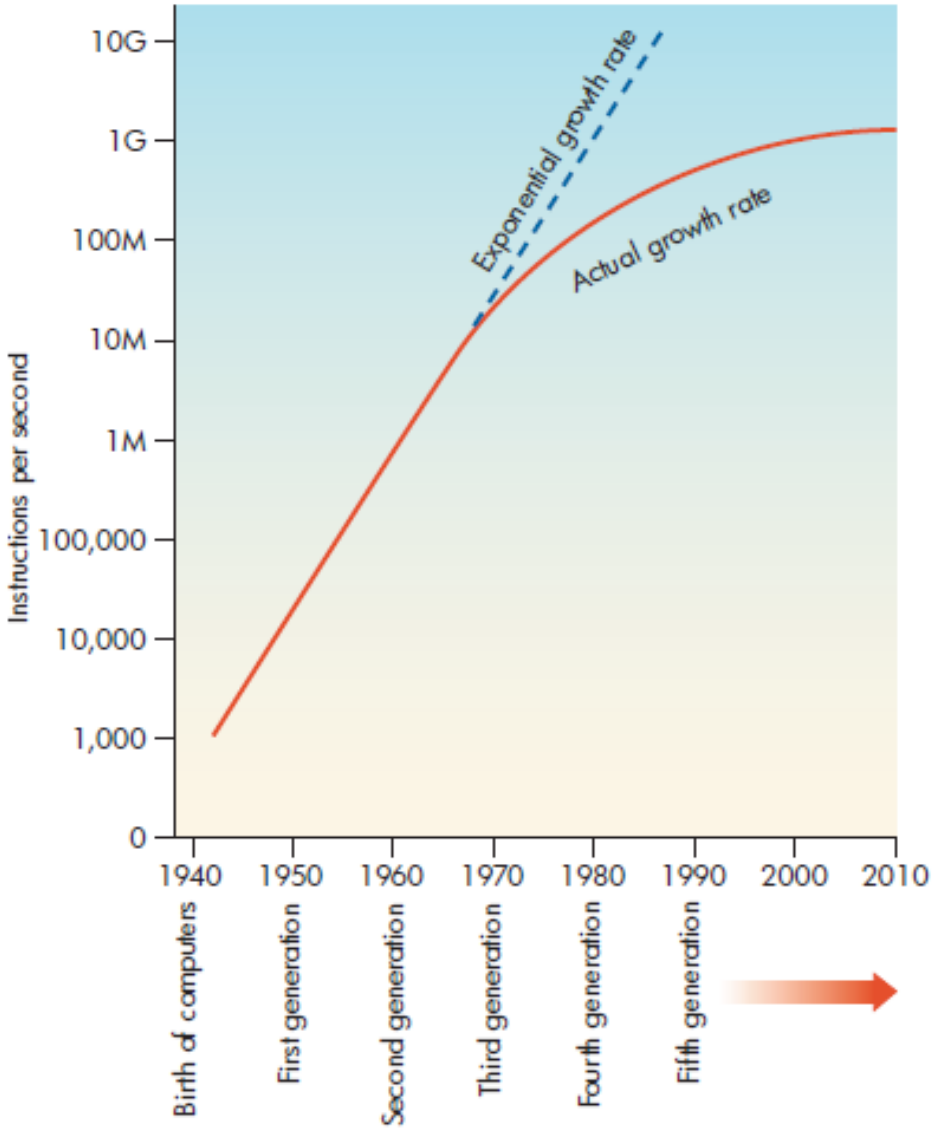


Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)

OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

Moore's Law


10G
1G
100M
10M
1M
100,000
10,000
1,000
0
Instructions per second
Exponential growth rate
Actual growth rate
1940
1950
1960
1970
1980
1990
2000
2010
Birth of computers
First generation
Second generation
Third generation
Fourth generation
Fifth generation

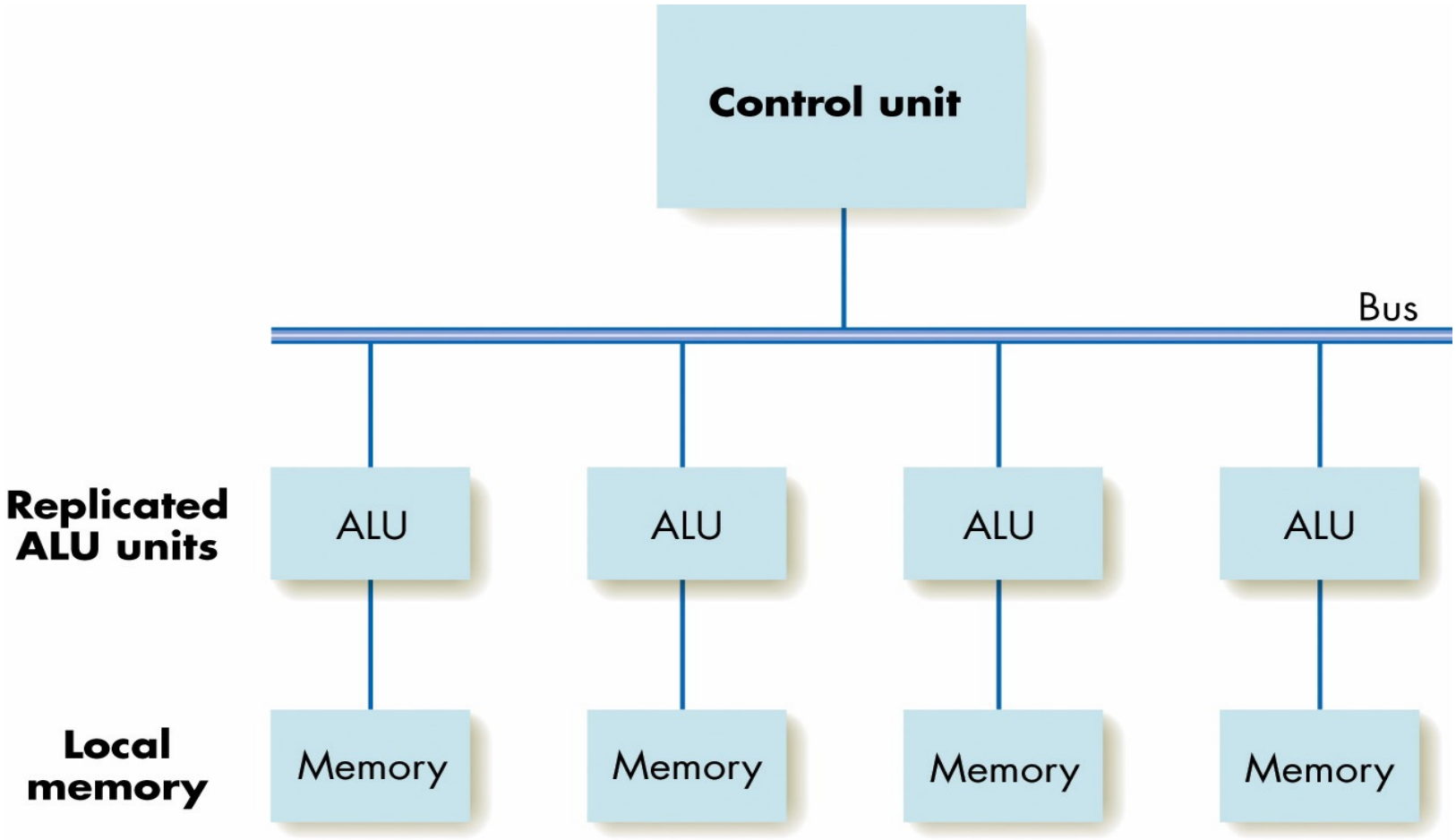
The Future

- Physical limitations on the speed of von Neumann computers
- Alternatives and derivatives of the von Neumann architecture explored to bypass these limitations
- Parallel computing architectures can provide improvements:
  - multiple operations occur at the same time



The Future

- SIMD architecture
  - Single instruction/Multiple data
  - Multiple processors running in parallel
  - All processors execute same operation at one time
  - Each processor operates on its own data
  - Suitable for “vector” operations

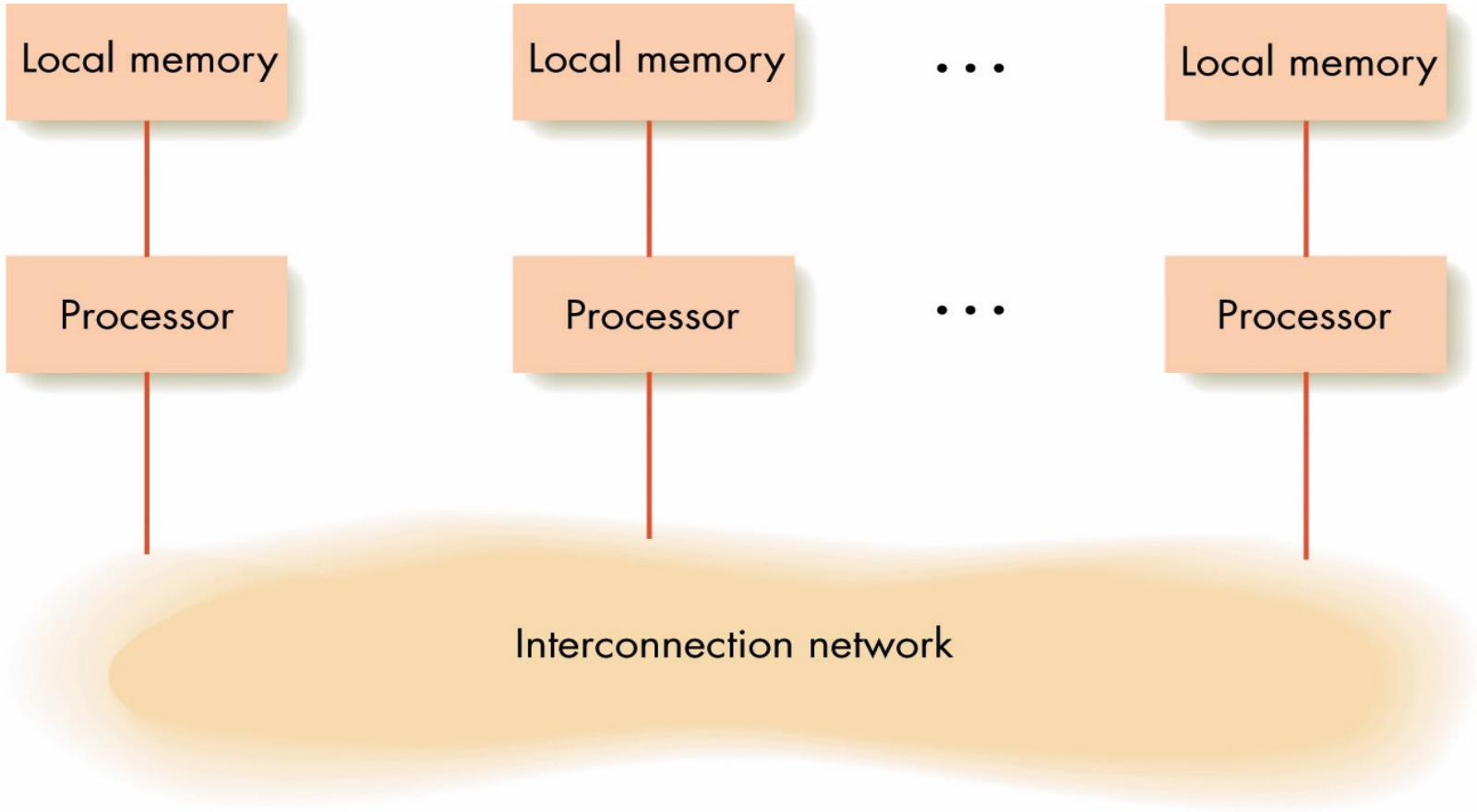


A SIMD Parallel Processing System

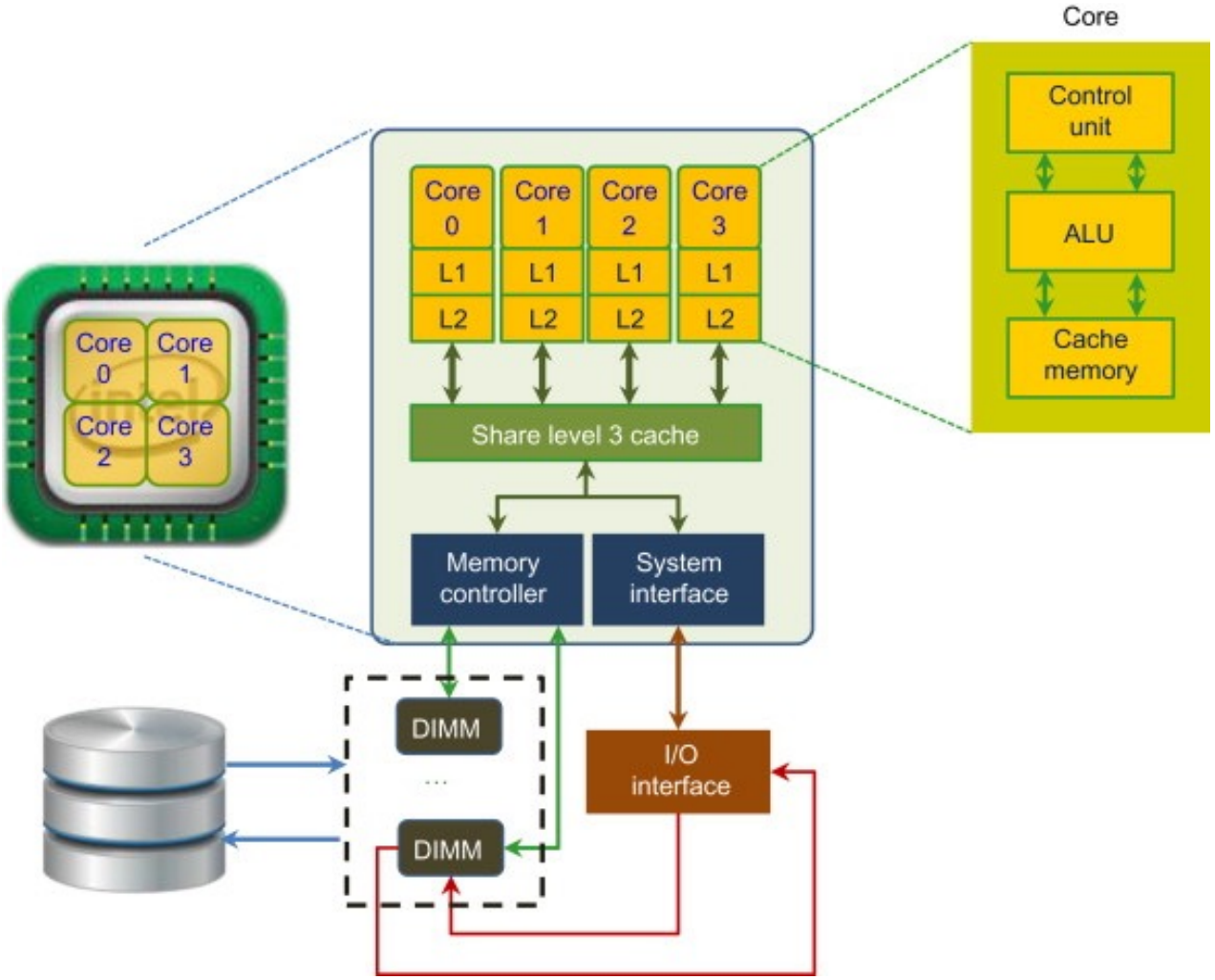


The Future

- MIMD architecture
  - Multiple instruction/Multiple data
  - Multiple processors running in parallel
  - Each processor performs its own operations on its own data
  - Processors communicate with each other



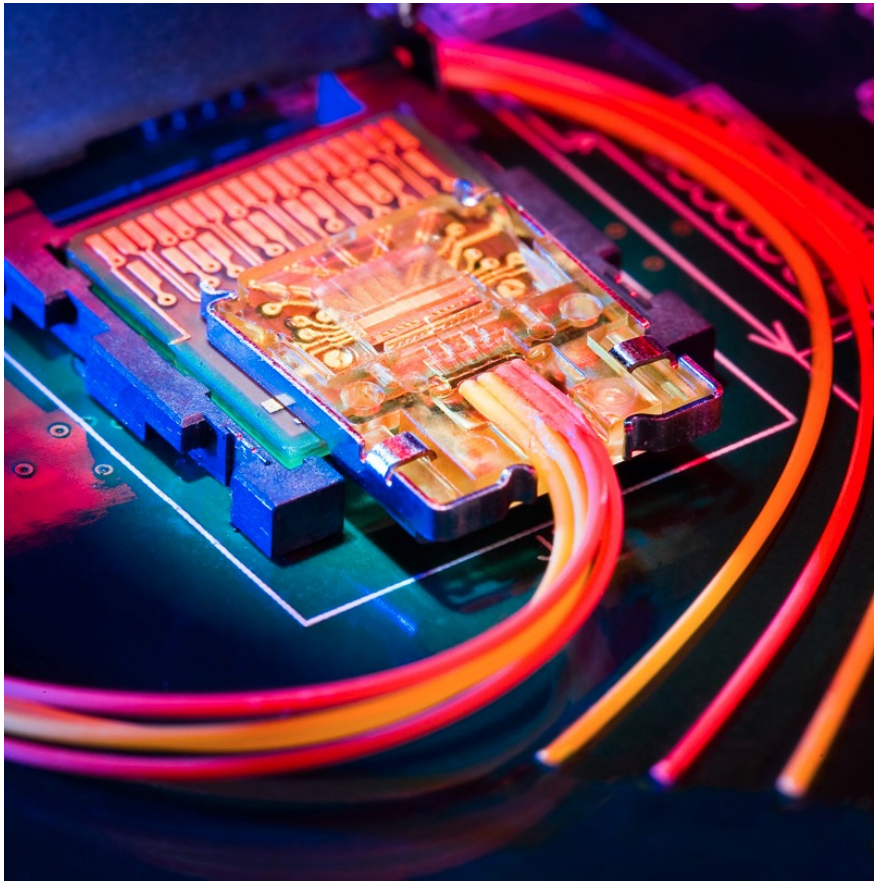
Model of MIMD Parallel Processing



https://www.sciencedirect.com/topics/computer-science/core-processor

New Trend: Optical Computing

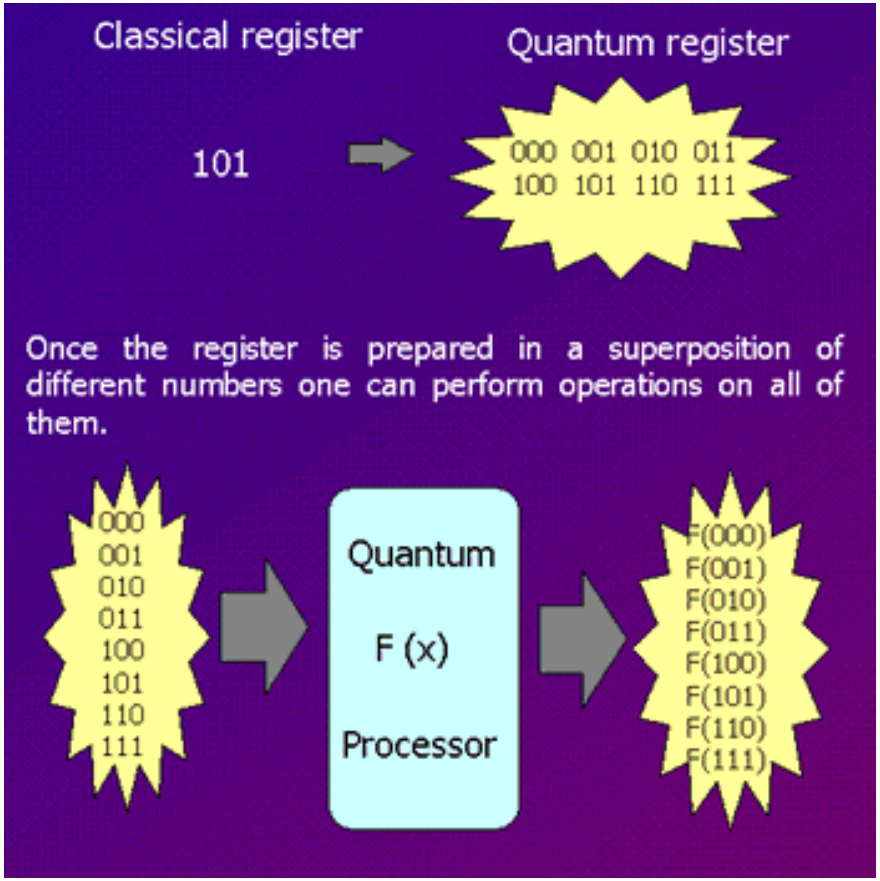
- Currently available for only data transfer.
- Work in progress towards “photonic logic” for designing circuits which use photons:
  - Many challenges



http://en.wikipedia.org/wiki/Optical_computing

New Trend: Quantum Computing





- There are several problems:
  - Decoherence: The more the qubits, the more their effect on the environment.

http://media.defenseindustrydaily.com/images/PUB_CQC_Cambridge_Quantum_Computing_Explained_lg.png

Read Especially This: http://www.cs.virginia.edu/~robins/The_Limits_of_Quantum_Computers.pdf



MORE ON MEMORY



Memory Types

- Dynamic Memory
  - The voltages stored in capacitors die away with time.
  - Solution?
    - Refresh the memory frequently; i.e., re-write the contents of the memory.
- Static Memory
  - A coupled transistor system stores the information.
  - One of the couples *triggers* the other.
- DRAM is cheaper and more widely used.

http://wiki.xtronics.com/index.php/How_Memory_Works



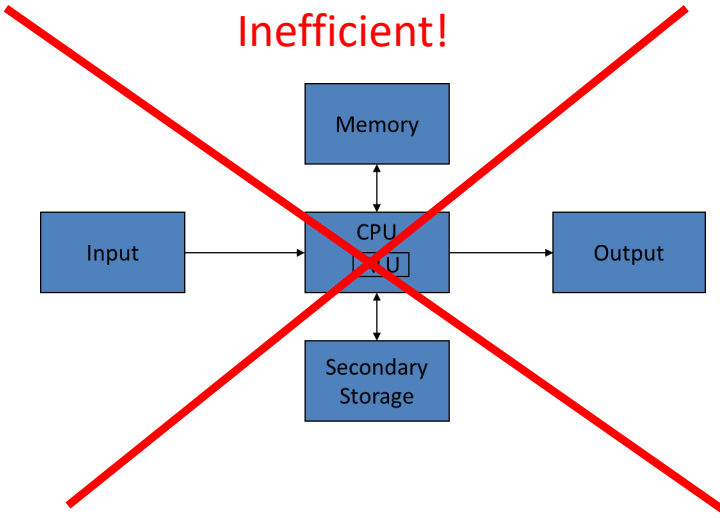
Memory Types

- Volatile Memory:
  - The stored values are lost when power is off.
  - DRAM, SRAM
- Non-volatile:
  - Read-only Memory (ROM), flash memory

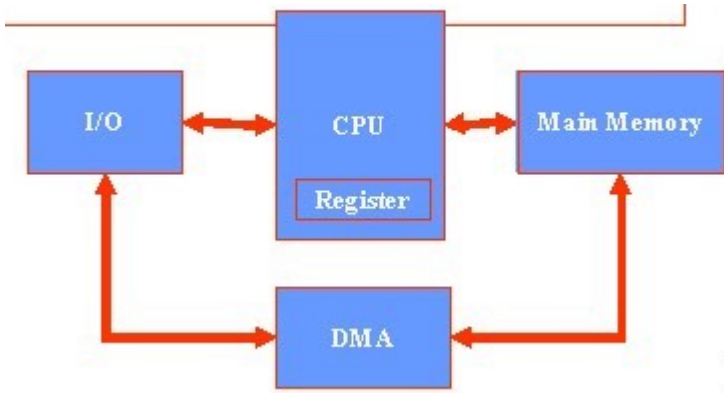
http://wiki.xtronics.com/index.php/How_Memory_Works

Direct Memory Access (DMA)

http://camnetway.blogspot.com/2009/11/interrupt-driven-io.html



- All memory accesses go over the CPU.
  - CPU gets the virtual address,
  - It looks up the physical address,
  - It writes the data to the physical device (memory, in most cases)
- This slows down the CPU for simple and consecutive memory accesses.
- "Modern" computers have a channel for DMA.
- The CPU just initiates the DMA command, and a separate unit (DMA controller) handles the copy of data between a device and a memory.





PERIPHERALS



I/O

- Memory-mapped I/O
  - Treats I/O devices as if they are memory
  - “Load” and “Store” instructions are used for accessing an I/O device
  - Single physical address space for memory and I/O devices
  - Devices: Graphics card, hard drive

- Port-mapped I/O (or isolated I/O)
  - Special instructions for performing I/O
  - Separate bus/pin for I/O devices
  - Devices: Keyboard, mouse, …

Input/Output and Mass Storage

- Communication with outside world and external data storage
  - *Input unit*
    - *"Receiving" section of computer*
    - *Obtains data from input devices*
      - *Usually a keyboard, mouse, disk or scanner*
    - *Places data at disposal of other units*
  - *Output unit*
    - *"Shipping" section of computer*
    - *Puts processed info on various output devices*
      - *Screens, paper printouts, speakers*
    - *Makes info available outside the computer*



Input/Output and Mass Storage

- Human interfaces: monitor, keyboard, mouse
- Archival storage: not dependent on constant power
- External devices vary tremendously from each other

Input/Output and Mass Storage (continued)

- Volatile storage
  - Information disappears when the power is turned off
  - Example: RAM
- Nonvolatile storage
  - Information does not disappear when the power is turned off
  - Example: mass storage devices such as disks and tapes

METU Computer Engineering

Input/Output and Mass Storage (continued)

- Mass *(secondary)* storage devices
  - Direct access storage device
    - Hard drive, CD-ROM, DVD, etc.
    - Uses its own addressing scheme to access data
  - Sequential access storage device
    - Tape drive, etc.
    - Stores data sequentially
    - Used for backup storage these days



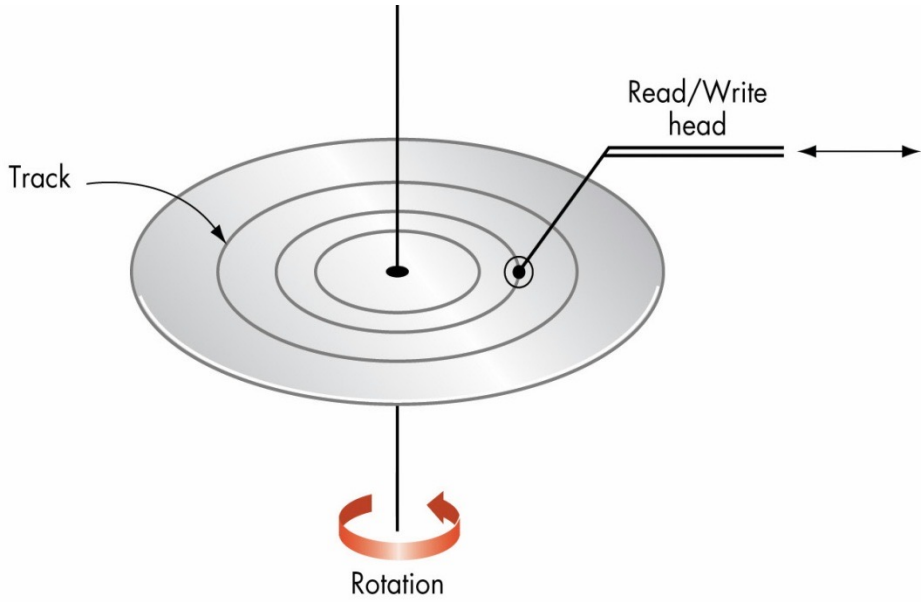
Input/Output and Mass Storage (continued)

- *Mass or Secondary storage unit*
  - Long-term, high-capacity “warehouse”
  - Stores programs or data not currently being used by other units on *secondary storage devices* (like discs)
  - Takes longer to access than primary memory
  - Data location

METU Computer Engineering

Input/Output and Mass Storage (continued)

- Direct access storage devices
- Data stored on a spinning disk
- Disk divided into concentric rings (tracks)
- Read/write head moves from one ring to another while disk spins
- Access time depends on:
  - Time to move head to correct track
  - Time for disc to spin to data location
- We will come back to rotational storage media!



METU Computer Engineering

The IBM 350

- 1956—the first hard disk drive
- 4.4 MB of storage
- Weighed over a ton
- About 1,000 systems built

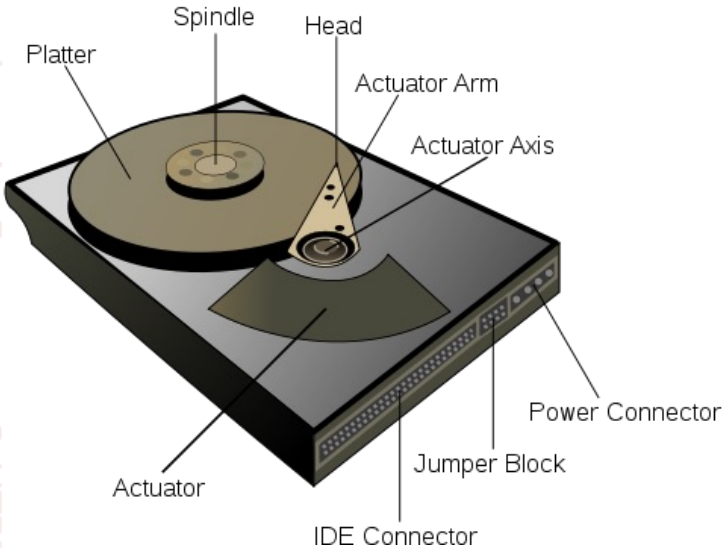


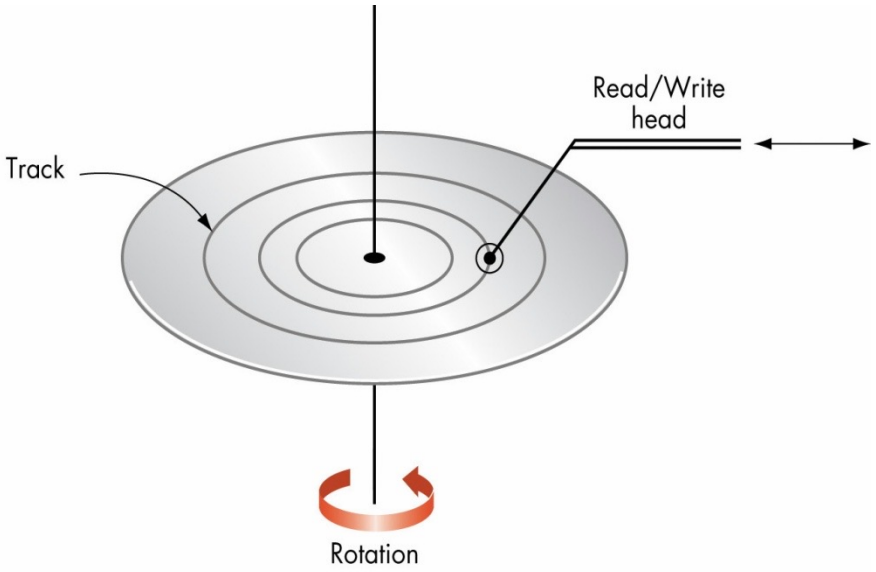
Image from www.ichigo.se/noterat/wp-content;
information from http://en.wikipedia.org/wiki/RAMAC

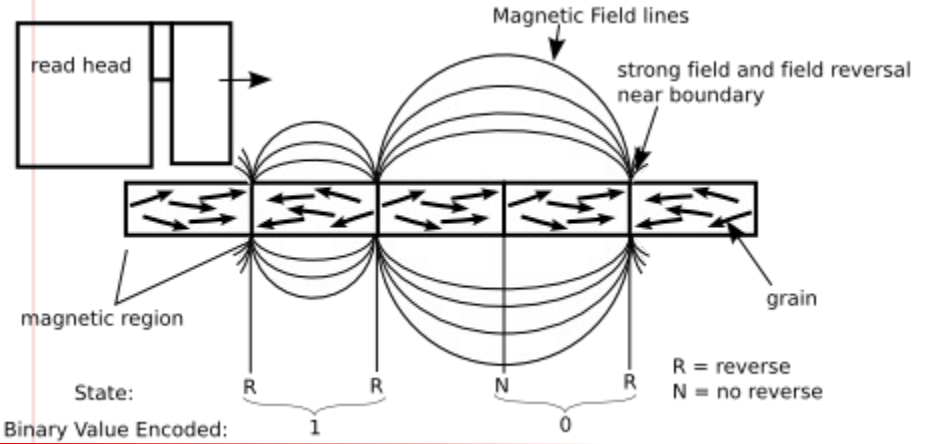
http://dwave.files.wordpress.com/2007/03/20070301_demo_slides.pdf

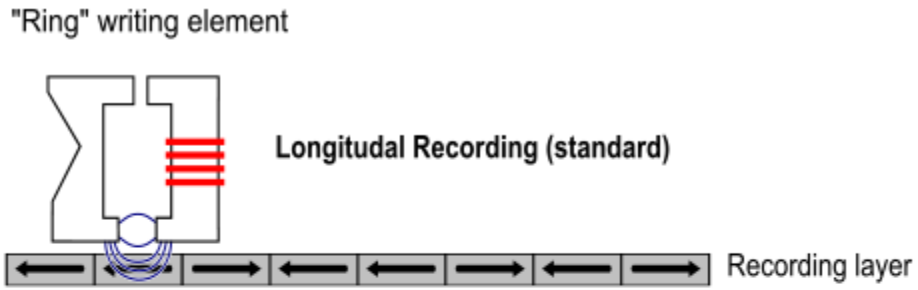
Rotational Media as External Storage

METU Computer Engineering



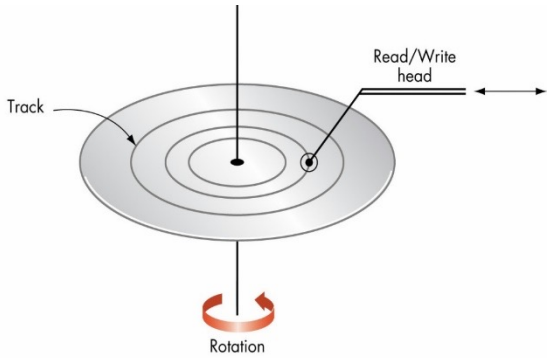






Rotational Drive Performance Characteristics

- For bigger disks, multiple layers & condensed layers are used.
- Access Time = Seek Time + Rotational Latency
  - Seek Time
    - "the time it takes the head to travel to the track of the disk where the data will be read or written"
    - around 9ms
  - Rotational Latency
    - "delay waiting for the rotation of the disk to bring the required disk sector under the read-write head."
    - Depends on Rotations Per Minute (RPM)
- Data Transfer Rate
  - Covers the internal rate & the external rate
  - SATA: 3.0 Gbit/s





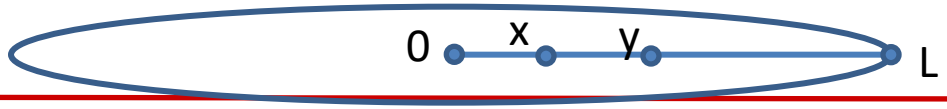
Note that alternative definitions of access time include data transfer rate as well. However, we will stick to this definition.

http://en.wikipedia.org/wiki/Disk_drive_performance_characteristics

One Last Bit of Information Regarding Rotational Media

- Average Seek Time
  - x: track that we want to move the arm to.
  - y: current position of the arm.
  - How can we model it in terms of the number of tracks?
- Average Seek Time = average distance $|x - y|$ multiplied by the time to move the arm by one track.
- The average distance = $L/3$. How?

Average Distance = $$\frac{1}{L^2}\int_0^L\int_0^L |x-y|\,dxdy = L/3$$



Solid State Disk (SSD)

- NAND Flash Memory Chips
- Access through the controller:
  - Faster than hard disk, still slower than RAM
- Better for daily usage
- Not so reliable for offline storage



Transfer of Information
The Concept of Ports

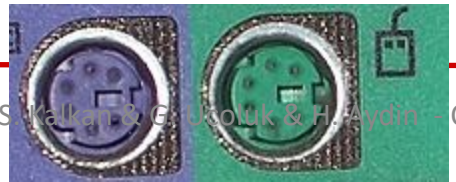
- What is a port, literally?
- What is a computer port, then?
  - SATA, e-SATA, USB, Firewire, Serial, Parallel (e.g., PS/2) ports.











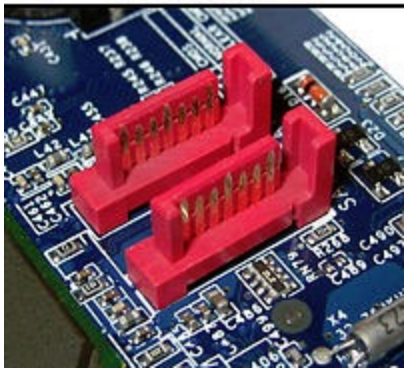




Transfer of Information In/On the Computer

- SATA – Serial Advanced Technology Attachment:
  - Bus interface between motherboard and storage devices, using high-speed cable.
  - Several versions (protocol & encoding is different):
    - SATA1: 1.5Gbit/s
    - SATA2: 3.0Gbit/s
    - SATA3: 6.0Gbit/s
- Alternatives:
  - IDE
  - PATA – Parallel Advanced Technology Attachment





Transfer of Information In/On the Computer

- Firewire – IEEE 1394 Interface
- High-Definition Audio-Video Network Alliance standard interface for audio/video communication.
- Several versions exist.
  - Around 800Mbit/s.
- Alternative:
  - Universal Serial Bus (i.e., USB)





Transfer of Information In/On the Computer

- Universal Serial Bus – USB
- Alternative to Firewire
  - Minor differences exist
- In addition to communication, USB can provide power to the device.
- Several versions exist: 1.0, 2.0, 3.0 and 4.0.
  - 2.0 is the current widely used standard.
  - 3.0 is available ~~but still not widely used~~.
  - 4.0 is becoming popular with the USB-C connector.



Transfer of Information Between Computers

- Ethernet
  - For inter-net as well as intra-net.
- Different versions:
  - 100Mbit/s
  - 1Gbit/s
- Ones that are not widely supported yet:
  - 10Gbit/s
  - 100Gbit/s







INTERRUPTS

* You are not responsible from this in the exam.

“Are we there yet?”



https://youtu.be/basofea2UEs?t=24

Interrupts

- When you type a character at the keyboard, the keyboard controller transmits the character to the CPU.
- However, the CPU is busy all the time and the keyboard controller has a limited storage; usually, only for one character.
- How to get the CPU's attention?
- Polling vs interrupt.
  - Polling: a phone without a bell.

http://www.atarimagazines.com/compute/issue149/60_Interrupts_made_easy.php



Interrupts

- I/O, time management, power-off signals, traps, …
- When an interrupt is received,
  - CPU stops the current task/program.
  - It saves the current “context”. How?
  - It executes the code necessary for the received interrupt.

Interrupt Table

- After booting, devices and operating systems register service routines that tell the computer "what to do" on different interrupts.
- The addresses of the routines that will handle the interrupts are stored in the interrupt table.

Interrupt vector	Description
00h	CPU: Executed after an attempt to divide by zero or when the quotient does not fit in the destination
01h	CPU: Executed after every instruction while the trace flag is set
02h	CPU: NMI, used e.g. by POST for memory errors
03h	CPU: The lowest non-reserved interrupt, it is used exclusively for debugging, and the INT 03 handler is always implemented by a debugging program
04h	CPU: Numeric Overflow. Usually caused by the INTO instruction when the overflow flag is set.
05h	Executed when Shift-Print screen is pressed, as well as when the BOUND instruction detects a bound failure.
06h	CPU: Called when the Undefined Opcode (invalid instruction) exception occurs. Usually installed by the operating system.
07h	CPU: Called when an attempt was made to execute a floating-point instruction and no numeric coprocessor was available.
08h	**IRQ0: Implemented by the system timing component; called 18.2 times per second (once every 55 ms) by the PIC**
09h	**IRQ1: Called after every key press and release (as well as during the time when a key is being held)**
0Bh	**IRQ3: Called by serial ports 2 and 4 (COM2/4) when in need of attention**
0Ch	**IRQ4: Called by serial ports 1 and 3 (COM1/3) when in need of attention**
0Dh	**IRQ5: Called by hard disk controller (PC/XT) or 2nd parallel port LPT2 (AT) when in need of attention**
0Eh	**IRQ6: Called by floppy disk controller when in need of attention**
0Fh	**IRQ7: Called by 1st parallel port LPT1 (printer) when in need of attention**

Timer Interrupt

- A system timer generates interrupts on every “tic”
- Called 18.2 times every second; once every 55ms.
  - Programmable
- Why is it needed?
  - Memory refreshing
  - OS time management
    - OS reads the initial time when booted
    - It increments the initial value with every time interrupt



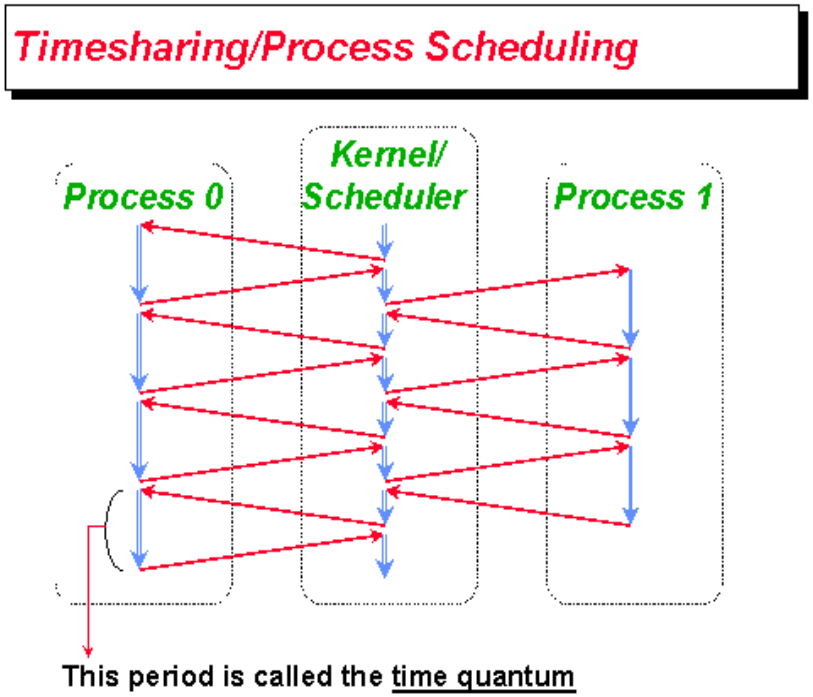
Masking Interrupts

- Disabling interrupts for a limited time.
- When is it needed?
  - Busy, handling an important interrupt already
  - Being interrupted by the interrupt being handled.

http://en.wikipedia.org/wiki/Interrupt_mask_register

Interrupts & Multi-tasking

- Most of the time, there is more than one program/process running.
- Each program runs for a fixed amount of time T.
  - T depends on the priority of the program/process.
- After T is finished, the CPU “interrupts”.



http://6004.csail.mit.edu/Spring98/Lectures/lect21/sld009.htm





What happens after that power button

BOOTING YOUR COMPUTER

* You are not responsible from this in the exam.

For more details, see e.g.: https://neosmart.net/wiki/mbr-boot-process/



Booting the Computer

- Basic Input-Output System (BIOS)
  - BIOS is on a Read-Only Memory (ROM)
  - When starting, CPU addresses the address space of the ROM and starts executing the BIOS.
  - BIOS first initiates self-check of hardware (Power-on self test – POST)
  - BIOS loads the rest of the BIOS into the RAM (because ROM is slow)
  - Then, it goes over non-volatile storage devices to find something bootable.
- Boot Device & Master Boot Record (MBR)
  - BIOS loads MBR from the boot device
  - MBR is not OS specific
  - MBR checks for a partition to boot
  - Loads OS (starts OS kernel)



Master Boot Record

- First 512-bytes of a partitioned data storage device; e.g., a disk.
- Responsibilities:
  - Holding the partition table
  - Helps booting of operating systems
  - Stores the ID of the disk (not used)
- A part of these 512-bytes is a code that searches the "primary" partitions and finds the one that is marked "active" in the table. The "volume boot record" of the active partition boots the OS.

Master Boot Record

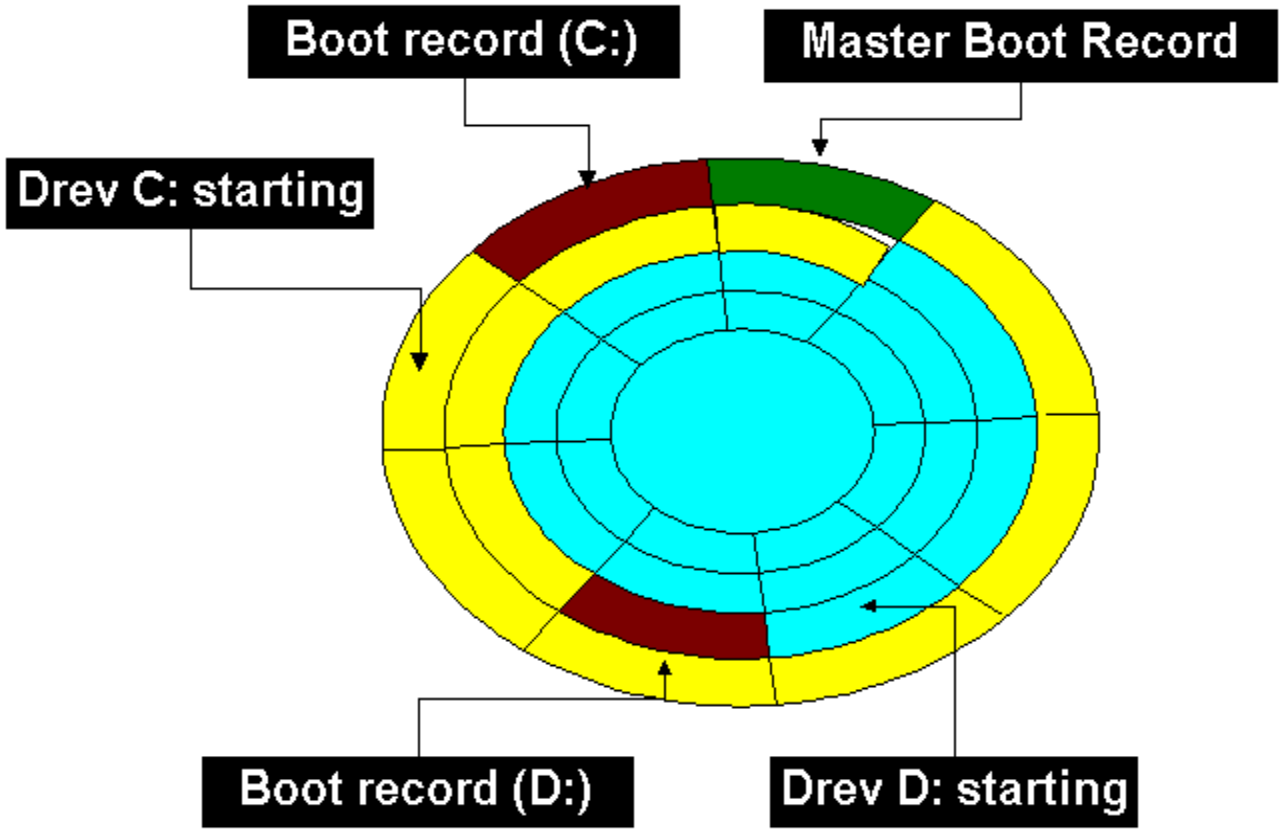


Fig: https://techstroke.com/

Master Boot Record

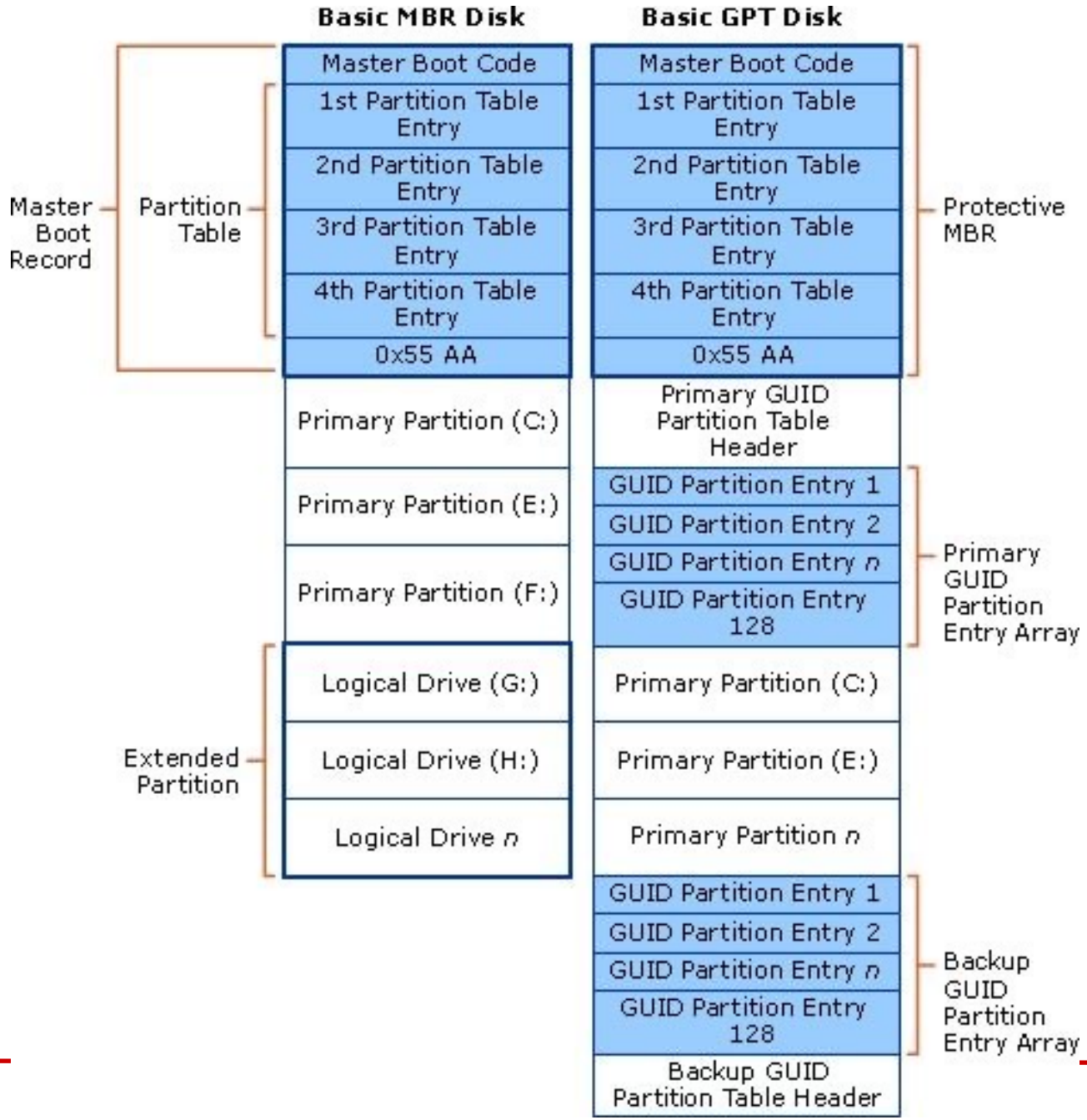
Structure of a Master Boot Record

Address			Description		Size in bytes
Hex	Oct	Dec			
0000	0000	0	code area		440 (max. 446)
01B8	0670	440	disk signature (optional)		4
01BC	0674	444	Usually nulls; 0x0000		2
01BE	0676	446	Table of primary partitions (Four 16-byte entries, IBM partition table scheme)		64
01FE	0776	510	55h	MBR signature; 0xAA55[1]	2
01FF	0777	511	AAh		
MBR, total size: 446 + 64 + 2 =					512



UEFI & GPT

- Unified Extensible Firmware Interface
  - Extends BIOS
  - managed by a group of chipset, hardware, system, firmware, and operating system vendors called the UEFI Forum
  - Faster hardware check compared to BIOS
  - Allows developers to add applications, enabling UEFI to be a lightweight OS
  - Provides secure boot, disables unauthorized applications from booting
  - Provides GUI
- GPT (GUID Partition Table) – required by UEFI
  - Extends MBR
  - GUID: Globally (Universally) unique identifier. Easily creatable almost unique identifier for partitions
  - Supports bigger disks (> 2TB), more partitions (up to 128)
  - It has backup at the end, which can help recover partitions



https://www.dell.com/community/Latitude/BIOS-setting-s-to-Install-Windows-Using-UEFI/td-p/7258597/page/3